

NGA MIS — Maintenance Plan

Teams, weekly cycle, quality gates and reporting · v1 · September 2026

The NGA MIS is four applications in four repositories, each deployed to production automatically from its main branch. From now on each module has its own team, and the whole system runs on a fixed weekly rhythm: **a plan at the start of the week, a progress report at the end**. This document is the operating agreement for that. It is short on purpose — the rules that matter are few, and everyone should know them.

Module	What it does	Repository	Production
Central MIS	Core school system and the sign-in provider (SSO) for the other three	nga_central_mis	mis.amashuri.com · api.amashuri.com
TaskMentor	Assignments, quizzes, grading, proctoring	nga-task-mentor	taskmentor.amashuri.com
Tendo	Discipline and attendance	nga-discipline-attendance	tendo.amashuri.com
Tupo	Communication: chat, mail, feed, meet	nga-communication-module	tupo.amashuri.com

1 · Teams and responsibilities

Role	Responsible for
Maintenance lead (one person, all modules)	Owns main in every repo: reviews and merges every Pull Request, watches every deploy, holds the rollback. Runs the Monday planning and Friday reporting cycle. Manages SSO registrations and secrets in Central MIS. Sends the consolidated weekly report upward.
Module team (one per module; names a module owner)	Everything inside their repository: bugs, features, tests, documentation, the module's weekly plan and report. Works on branches, never on main. Raises anything cross-module (see §6) with the lead before starting.
Module owner (one per team)	First reviewer on the team's PRs, keeps the module backlog in order, writes the team's weekly plan and progress report, is the contact for incidents in that module.

2 · Ground rules (non-negotiable)

- **main is production.** Every push to main deploys within minutes. Nobody commits to main directly; it changes only by merging a reviewed PR.
- **Every change is a branch + Pull Request** (feature/..., fix/...), reviewed by the module owner, merged by the lead.
- **Every change is tested locally first** — the module's setup guide in guides/ gets any developer running in under 30 minutes.
- **One repo deploys at a time**, and someone watches it finish and checks production afterwards (§5).
- **No secrets in git.** .env files stay local; templates (.env.example) are what gets committed. Client secrets come from the lead.
- **Database changes ship as migration files** in the repo, never as hand edits on the server (§6).

3 · Before feature work: onboarding and the initial audit

Nobody plans features for a system they have not used. The first two weeks are fixed:

When	Team does	Deliverable
Week 1 — Onboarding	Each developer sets up their module locally using its guide (guides/<Module>-Local-Setup-Guide.pdf), signs in, and clicks through every screen. The lead provides MIS accounts, the module's SSO client secret, and whitelists the local callback URLs.	Every developer can run their module and sign in. Blockers reported to the lead by Wednesday.
Week 1–2 — Audit	The team goes through the whole module as a user would (every role: admin, teacher, student where applicable) and records, in GitHub Issues on the module's repo, every bug , usability problem , missing feature and technical risk they find. One issue per item, using the template in Appendix A.	Audit report (Appendix A summary table) by end of week 2, plus a labelled, prioritised backlog in GitHub Issues.
End of week 2 — Planning baseline	Lead and module owners review all four audit reports together, agree severity and priority across modules, and pick the first week's work.	The first weekly plan (§4) for each module.

Labels every repo uses: bug · feature · ux · tech-debt · security and a severity S1 (production broken / data at risk) · S2 (major function broken, workaround exists) · S3 (minor) · S4 (cosmetic / nice-to-have).

4 · The weekly cycle

Day	What happens	Owner
Monday	Planning: each team fills the Weekly Plan (Appendix B) from its backlog — at most 3–5 items, each with an owner, an estimate and a “done” definition. 30-minute planning call, lead + module owners. Plans posted in the shared folder by noon.	Module owners → lead
Tue–Thu	Work on branches. PRs opened as soon as an item is ready; module owner reviews within one working day. Short daily check-in inside each team (async is fine).	Teams
Thursday	Merge window. Lead merges reviewed PRs one repo at a time, watches each deploy, verifies production (§5). Nothing merges on Friday afternoon — if it is not in by Thursday, it waits for next week.	Lead
Friday	Reporting: each team fills the Weekly Progress Report (Appendix C): done / not done and why / issues found / next week's candidates. Lead consolidates the four into one report and sends it upward by end of day.	Module owners → lead → management

Estimates are in half-days. A plan item that will take more than a week is split. A plan that changes mid-week is updated, not silently abandoned — the Friday report explains the change.

5 · Definition of done, and how a change reaches production

A Pull Request is ready to merge when **all** of these are true:

- It does one thing, described in the PR title and body (what, why, and “**production impact**” in one line).
- It was run locally by the author; tests pass (npm test where the module has them); no .env, node_modules, uploads or build output in the diff.
- Any database change is a migration file, applied on the author's local DB, with a rollback noted in the PR.

- The module owner reviewed it (a second person for anything touching sign-in, permissions, grading, attendance records or money).

The lead then merges and runs this checklist — every time, no exceptions:

Step	Detail
1. Merge	GitHub “Merge pull request” on the reviewed PR. Note the merge commit SHA.
2. Watch	GitHub → Actions → the dep loy run. Wait for green. If red: production is untouched — read the log, fix forward or re-run.
3. Verify	Open the module in production and hit its health URL: <code>api.amashuri.com/health</code> , <code>taskmentor-api.amashuri.com/health</code> , <code>tendo.amashuri.com/health</code> , <code>tupo.amashuri.com/svc/api/health</code> . Sign in once.
4. Roll back if needed	<code>git revert -m 1 <merge sha></code> on main, push. That triggers a normal deploy of the previous state — about three minutes, no force-push, ever.
5. Record	One line in the week’s report: what shipped, when, verified by whom.

6 · Cross-module rules

- **Central MIS is upstream of everything.** TaskMentor, Tendo and Tupo authenticate through it and read rosters, users and permissions from its API. Any change to `/sso/*`, `/auth/*`, `/users/*`, `/academics/*` or the System table is announced to all module owners *before* the PR is opened, and deployed first in the merge window.
- **Schema changes are migrations.** Central MIS: numbered SQL files in `backend/migrations/`; TaskMentor: Sequelize migrations; Tupo: `packages/db` migrations. A migration is applied to production by the deploy or the lead — never by hand in a database client.
- **Shared identifiers are frozen** unless all owners agree: SSO client ids, callback URLs, the health endpoints, port numbers in the local setup.
- **One change, one repo.** A feature that needs two repositories is two PRs with the dependency stated, deployed in dependency order.

7 · When production breaks

Severity	Means	Response
S1	Users cannot sign in, data is being lost or exposed, a module is down	Lead is called immediately. First action is rollback (§5 step 4) unless the cause is obviously elsewhere. Fix follows on a branch. Written incident note in Friday’s report.
S2	A major function is broken but there is a workaround	Issue opened with S2, fix goes to the top of this week’s plan, ships in the next merge window (or sooner if the lead calls it).
S3 / S4	Minor or cosmetic	Backlog; scheduled in a normal weekly plan.

Always check the deploy log and the health URL before assuming code is the cause — a failed `npm ci` or a full disk looks like an outage too.

8 · The lead's week at a glance

Monday	Collect four weekly plans · run planning call · check every plan has owners, estimates and “done” definitions · confirm nothing cross-module is unannounced
Tuesday–Thursday	Review PRs as they come · keep the merge queue short · Thursday: merge, watch, verify, one repo at a time
Friday	Collect four progress reports · verify each “done” item is actually in production · consolidate · send upward · update next week's candidates
Monthly	Rotate/verify SSO secrets are still only in the right hands · review open S1/S2 count · check backups exist and restore once · prune stale branches

Appendix A · Audit issue and audit report

One GitHub Issue per finding. Title format: [Module] Short description — e.g. [Tendo] Late-arrival report shows wrong week.

Audit issue (GitHub Issue body)	
Where	screen / URL / role you were signed in as
What happens	exact steps, then what you saw (screenshot or copied error)
What should happen	expected behaviour, or the feature as the user would describe it
Type · Severity	bug / feature / ux / tech-debt / security · S1–S4
Notes	suspected cause, related issues, anything the lead should know

The audit report is one table per module, sent to the lead at the end of week 2:

Audit report — <Module>, <date>, <team members>	
Screens reviewed	list every screen / role covered; note anything not reachable
Findings	count by type and severity, e.g. bugs: 2×S1, 5×S2, 9×S3 · features: 4 · ux: 6 · tech-debt: 3
Top 5 by impact	issue numbers with one line each — what the team thinks should ship first
Risks	anything that could break production or lose data, even if not a bug yet
Setup problems	anything in the local setup guide that was wrong or missing

Appendix B · Weekly plan (Monday)

Weekly plan — <Module> — week of <date>	
Goal for the week	one sentence — what will be true on Friday that is not true today
Item 1	issue # · owner · estimate (half-days) · done = <how we will know it is finished>
Item 2	...
Item 3	... (3–5 items maximum)
Cross-module	anything that touches Central MIS or another module — announced to whom, when
Risks / needs	what could block this, and what the team needs from the lead (access, secrets, decisions)

Appendix C · Weekly progress report (Friday)

Weekly progress report — <Module> — week of <date>	
Done	items shipped to production this week: issue # · PR # · deployed on <date> · verified by <name>
Not done	planned items not shipped, each with a one-line reason and where it stands (in review / blocked / re-estimated)

Found this week	new issues opened, by severity; any S1/S2 with a one-line summary
Production	incidents (or “none”), rollbacks (or “none”), health checks passed
Next week candidates	3–5 items the team proposes for Monday's plan
Asks	decisions or help needed from the lead or management

The lead's consolidated report is the four module reports stacked, with a two-line summary on top: *what shipped, what is at risk*.